



Java

Clase 04 | Funciones y Modularización

Índice

Clase 04

Clase 4 | Funciones y Modularización

- Concepto de funciones (métodos) en Java.
- Declaración y llamada de funciones.
- Parámetros y argumentos.
- Paso por valor y referencia.
- Sobrecarga de métodos.
- Buenas prácticas de modularización.

Clase 05

Clase 5 | Programación Orientada a Objetos (POO)

- Paradigmas de programación: estructurada vs. POO.
- Clases, objetos y atributos.
- Creación de clases y objetos en Java.
- Métodos de instancia y de clase.
- Constructores y métodos especiales.

Objetivos de la Clase

1

Entender funciones

Comprender qué son las funciones en Java y cómo se diferencian de otras partes del código.

3

Paso por valor

Entender el concepto de paso por valor y su impacto en el comportamiento de los métodos.

2

Declarar y llamar métodos

Aprender a declarar y llamar métodos, entendiendo parámetros y argumentos.

4

Sobrecarga y modularización

Conocer la sobrecarga de métodos y aplicar buenas prácticas de modularización.

Funciones

Introducción a Funciones en Java

Las funciones, también conocidas como métodos en Java, son bloques de código reutilizables que realizan una tarea específica. Funcionan como pequeñas "máquinas" que pueden recibir datos, procesarlos y devolver un resultado.

Las funciones nos permiten:

- Organizar mejor nuestro código dividiéndolo en partes más pequeñas y manejables
- Reutilizar código evitando la repetición
- Hacer nuestros programas más fáciles de mantener y entender



Ejemplo simple de una función:

```
// Función que suma dos números
public static int sumar(int a, int b) {
    return a + b;
}

// Uso de la función
int resultado = sumar(5, 3); // resultado = 8
```

En este ejemplo, la función 'sumar' recibe dos números como parámetros, realiza la operación y devuelve el resultado. Esta función puede ser reutilizada en cualquier parte del programa donde necesitemos sumar dos números.

Características de las funciones en Java



Reutilización

El código de una función se puede usar en diferentes partes del programa.



Modularidad

Divide el programa en partes más pequeñas y manejables.



Abstracción

Esconde la complejidad del código interno de la función.



Programación con Funciones vs. Programación Lineal

Programación Lineal

El código se ejecuta de forma secuencial, línea por línea. Esto puede ser difícil de leer y mantener, especialmente para programas largos y complejos.

Programación con Funciones

El código se divide en bloques reutilizables, llamados funciones. Esto hace que el código sea más legible, mantenible y fácil de probar.



Declaración de Funciones

Una función en Java se define con una firma que incluye el tipo de retorno, nombre y lista de parámetros.

```
public int sumar(int a, int b) {  
    int resultado = a + b;  
    return resultado;  
}
```

Componentes de una Función en Java

```
public int calcularTotal(int precio, int cantidad) {  
    return precio * cantidad;  
}
```

1. Modificador de Acceso (public)

Determina la visibilidad de la función desde otras partes del programa. En este caso, 'public' permite que la función sea accesible desde cualquier parte.

2. Tipo de Retorno (int)

Especifica el tipo de dato que la función devolverá. Aquí, 'int' indica que devolverá un número entero.

3. Nombre (calcularTotal)

Identifica la función y describe su propósito. El nombre 'calcularTotal' indica claramente que calcula un total.

4. Parámetros (precio, cantidad)

Son los valores que recibe la función para realizar su tarea. En este ejemplo, necesita el precio y la cantidad para calcular el total.

Declaración y Llamada de Funciones

1

Declaración de la Función

Se define la función en el cuerpo de la clase, fuera de otros métodos. Aquí especificamos el nombre, parámetros y tipo de retorno.

2

Llamada a la Función

Se invoca la función desde el método main o desde otros métodos, proporcionando los argumentos necesarios que coincidan con los parámetros definidos.

3

Ejemplo Práctico

```
double precioConDescuento =  
    aplicarDescuento(precioOriginal,  
        15.0); // Llama a la función pasando el  
        precio y porcentaje de descuento
```

Parámetros y Argumentos



Parámetros

Variables declarativas que actúan como placeholders en la definición del método. Por ejemplo, en "void setNombre(String nombre)", "String nombre" es el parámetro que especifica qué tipo de dato espera recibir el método.

Argumentos

Valores reales que se envían al método durante su ejecución. Por ejemplo, cuando escribimos "setNombre("Juan")", "Juan" es el argumento que se asigna al parámetro nombre. Los argumentos deben coincidir con el tipo de dato especificado en los parámetros.

Ejemplo de una función con parámetros

Esta función aplica un descuento especial que varía según la cantidad de productos comprados. Los parámetros permiten flexibilidad en el cálculo:

1 Descuento Base

Aplica el porcentaje de descuento inicial especificado en el parámetro 'porcentaje'

2 Descuento por Volumen

Si la cantidad supera 50 unidades, añade automáticamente un 5% adicional al descuento base

```
public static double aplicarDescuentoConCantidad(double precio, double porcentaje, int cantidad)
{
    double descuentoFinal = porcentaje;
    if (cantidad > 50) {
        descuentoFinal = porcentaje + 5;
    }
    double descuento = precio * (descuentoFinal / 100);
    return precio - descuento;
}
```

Problema: Imprimir un recibo

En la tienda de café Sibelius necesitan que el sistema pueda imprimir recibos detallados para sus clientes, mostrando tanto los precios originales como los descuentos aplicados en sus diversos productos de café y repostería.

Para solucionar esta necesidad, se implementa un método que requiere:

- producto: El nombre del producto de café o repostería vendido
- precioOriginal: El precio base del producto antes de aplicar promociones
- precioFinal: El precio después de aplicar el descuento correspondiente

Este método automatiza la generación de recibos, asegurando que cada cliente reciba un comprobante claro que muestra el detalle de su compra, incluyendo el nombre del producto seleccionado, su precio original y el precio final con descuento aplicado.

► [Hacé clic acá para ver la solución](#)

- producto: El nombre del producto de café o repostería vendido
- precioOriginal: El precio base del producto antes de aplicar promociones
- precioFinal: El precio después de aplicar el descuento correspondiente

Este método automatiza la generación de recibos, asegurando que cada cliente reciba un comprobante claro que muestra el detalle de su compra, incluyendo el nombre del producto seleccionado, su precio original y el precio final con descuento aplicado.

▼ **Hacé clic acá para ver la solución**

```
public static void imprimirRecibo(String producto, double precioOriginal, double
precioFinal) {
    System.out.println("Recibo de Compra");
    System.out.println("Producto: " + producto);
    System.out.println("Precio Original: $" + precioOriginal);
    System.out.println("Precio con Descuento: $" + precioFinal);
}
```

Este método permite a main enfocarse en la orquestación y no en los detalles.

Problema: Cálculo de Descuentos

Contexto

Sibelius, una tienda de Café en expansión, necesita implementar un sistema flexible de descuentos para su diversa gama de productos. El desafío principal es mantener la consistencia en los cálculos y evitar la duplicación de código en diferentes partes del sistema.

Tip

Desarrollar un método reutilizable llamado `aplicarDescuento` que reciba como parámetros el precio base del producto y el porcentaje de descuento. Este método encapsulará la lógica del cálculo y retornará el precio final con el descuento aplicado, garantizando precisión y consistencia en todas las operaciones.

► [Hacé clic acá para ver la solución](#)

Contexto

Sibelius, una tienda de Café en expansión, necesita implementar un sistema flexible de descuentos para su diversa gama de productos. El desafío principal es mantener la consistencia en los cálculos y evitar la duplicación de código en diferentes partes del sistema.

Tip

Desarrollar un método reutilizable llamado `aplicarDescuento` que reciba como parámetros el precio base del producto y el porcentaje de descuento. Este método encapsulará la lógica del cálculo y retornará el precio final con el descuento aplicado, garantizando precisión y consistencia en todas las operaciones.

▼ [Hacé clic acá para ver la solución](#)

```
public static double aplicarDescuento(double precio, double porcentaje) {  
    double descuento = precio * (porcentaje / 100.0);  
    double precioFinal = precio - descuento;  
    return precioFinal;  
}  
//Este método se puede invocar múltiples veces con diferentes precios y descuentos.
```

Sobrecarga de Métodos

Definición

La sobrecarga de métodos es una técnica de programación que permite crear múltiples versiones de un método con el mismo nombre pero diferentes parámetros. Esto facilita la reutilización de código y mejora la legibilidad del programa.

Utilidad

Esta técnica aumenta la flexibilidad del código permitiendo que un mismo método procese diferentes tipos o cantidades de datos. Por ejemplo, un método `calcularArea()` puede aceptar tanto un radio para círculos como base y altura para rectángulos, manteniendo una interfaz consistente.

Ejemplo de Sobrecarga de metodos

Estos métodos permiten calcular descuentos de forma flexible: uno básico con porcentaje y otro que incluye un bonus fijo adicional para clientes premium.

```
// Descuento básico con porcentaje
public static double aplicarDescuento(double precio, double porcentaje) {
    double descuento = precio * (porcentaje / 100.0);
    return precio - descuento;
}

// Descuento con bonus fijo adicional
public static double aplicarDescuento(double precio, double porcentaje, double bonusFijo) {
    double descuento = precio * (porcentaje / 100.0);
    return precio - descuento - bonusFijo;
}
```

Buenas Prácticas de Modularización



Una responsabilidad

Cada método debe realizar una sola tarea concreta.



Nombres descriptivos

Los nombres de los métodos deben reflejar su función.



Métodos concisos

Evitar métodos demasiado largos, dividiéndolos si es necesario.



Reutilización

Código modular es más fácil de mantener, probar y extender.

Recursos Adicionales

Documentación oficial

Consulta la documentación de Java para profundizar en los conceptos.



docs.oracle.com



Defining Methods (The Jav...

This beginner Java tutorial describes fundamentals of programming in the Java...

Guía de métodos

Revisa la guía de Oracle sobre métodos en Java.

<https://docs.oracle.com/javase/8/docs/api/java/lang/reflect/Method.html>

Videos recomendados

Mira "Organización del Código con Métodos" en YouTube.

https://youtube.com/playlist?list=PL954bYq0HsCXlwPGM2k2nxRVztHOwpPqc&si=GfIFagrxE_BjMaw4

Reflexiones Finales



Facilidad de lectura

¿Cómo las funciones mejoran la lectura del código en proyectos grandes?



Sobrecarga

¿Cómo la sobrecarga te permite manejar diferentes escenarios?



Beneficios

¿Qué ventajas trae la modularización al agregar o modificar funcionalidades?



Ejercicios prácticos

Situación inicial en TechLab.



Silvia, la Product Owner, acaba de asignar una nueva tarea al equipo de desarrollo: el sistema debe aplicar distintos tipos de descuentos a los productos según la categoría, la cantidad comprada y el tipo de cliente. Matías y Sabrina se dan cuenta de que el código está creciendo y volviéndose complejo. Necesitan una forma de organizar el código en bloques más pequeños y reusables: las funciones.

Además, habrá escenarios donde será necesario crear métodos con el mismo nombre pero diferentes parámetros, gestionar el impacto de las variables dentro de los métodos y mantener un código claro y fácilmente ampliable.

Ejercicio Práctico

Optativo | No entregable

Para poder llegar con los tiempos de entrega dispuestos por Silvia, será necesario realizar las siguientes acciones:

Creación de Funciones

- Escribí un método `calcularImpuesto` que reciba un precio y un porcentaje de impuesto, y devuelva el precio final.
- Llamá al método desde `main` con distintos valores.

Parametros y argumentos:

- Creá un método `calcularPrecioFinal` que reciba el precio, el descuento y la cantidad. Si la cantidad es mayor a 50, aumentá el descuento. Imprimí el resultado.
- Definí un array con el stock de 3 productos.
- Escribí un método `reponerStock` que sume unidades a un índice específico del array.
- Mostrá el stock antes y después de la reposición.

Ejercicio Práctico

Optativo | No entregable

Sobrecarga de métodos:

- Creá dos métodos mostrarMensaje: uno recibe una String, otro recibe una String y un int.
- Usá ambos para imprimir mensajes en función de si el cliente es nuevo o recurrente.

Modularización:

- Tomá un código previo (como el de la clase anterior) y extraé partes en métodos.
- Comentá por qué ahora es más fácil de leer y mantener.